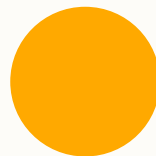




Javascript Loops

Reese Hatfield



0





Loops

- Take a look at the tools we have:
 - Variables
 - Math operations
 - Basic Input and Output
 - Conditionals
 - Dynamic rendering





Loops

- These tools can enable us to solve problems
- Most* problems at least
- If Statements let us branch out our program
- But we are still limited in our approach





Loops

- Let's consider a new problem:
- "Receipt Totaller"
- Have my user:
 - Enter everything they bought at the store (via price)
- Show them a simple website
 - Tell them how much they spent





Loops

- Break the problem down
 - Need to prompt
 - Store the sum of the all their items
- Let's begin programming
 - Test as we go





Loops

- Problem:
- "How many items did the user buy?"
- This controls how many times we need to prompt() them.
- We need to introduce a new programming concept for this





Loops

- Enter **Loops**
- Loops let you execute any block of code
 - Until a conditional is/isn't met
- Several kind of loops





Loops

- First type
- While Loop
- Simple syntax

*while(conditional) {
 expressions;*





Loops

- All the code within the {}s will execute while the conditional is TRUE
- Once we reach the bottom, go up to the top, and restart

```
while(conditional) {  
    expressions;  
    ...  
}
```





Loops

- Pattern:
 - Check Conditional
 - True $\Rightarrow$ Execute block
 - Repeat
 - False $\Rightarrow$ Exit loop

```
while(conditional) {  
    expressions;  
    ...  
}
```





Loops

- Take a look at the following code:
- What happens?
- Let's find out

```
while(true) {  
    alert("hello user");  
}
```





Loops

- Alert line will run while *true is true*
- True always is true
- Constantly execute

```
while(true) {  
    alert("hello user");  
}
```





Loops

- More common pattern:
- Update some variable
- Use that variable as the conditionals

```
var n = 0;
while(n < 3) {
    alert("hello user");

    n = n + 1;
}
```

+
○ ● How many times?



Loops

- $0 < 3 \Rightarrow \text{True}$
- $1 < 3 \Rightarrow \text{True}$
- $2 < 3 \Rightarrow \text{True}$
- $3 < 3 \text{ ???}$

```
var n = 0;  
while(n < 3) {  
    alert("hello user");  
  
    n = n + 1;  
}
```





Loops

- $0 < 3 \Rightarrow \text{True}$
- $1 < 3 \Rightarrow \text{True}$
- $2 < 3 \Rightarrow \text{True}$
- $3 < 3 \Rightarrow \text{False}$

Code will execute only three times, **print n**

```
var n = 0;
while(n < 3) {
    alert("hello user");

    n = n + 1;
}
```





Loops

- Using these tools:
 - We should be better equipped to solve our receipt problem
- Let's give it another shot!





Loops

- One more problems:
- While loops let us re-run a slice of our code an arbitrary amount of times
- We still don't quite know when our user is finished typing





Loops

- When repeatedly asking for input:
 - A common pattern is using a "sentinel value"
- Let me know when you are finished typing
- Let's use "-1"





Loops

- Take a look at this old loop that we had
- Key difference
- Do you know how many times a loop will run?

```
var n = 0;  
while(n < 3) {  
    alert("hello user");  
  
    n = n + 1;  
}
```

Depends





Loops

- This kind of loop
- Where you know *ahead* of time how many times it will run is a very common pattern
- +
 - Declare, check, execute, update

```
var n = 0;  
while(n < 3) {  
    alert("hello user");  
  
    n = n + 1;  
}
```



Loops

- If you ever run into this (happens more often than not)
- Special type of loop that is built for this pattern
- Called the ***for loop***
- Let's dive into that pattern





Loops

- Different syntax

```
for(declaration; conditional; update){  
    expressions;  
    ...  
}
```

- Four pieces, just like in our initial case





Loops

- 1. Declare new variable
- 2. Conditional (based on said variable)
- 3. Update variable
- 4. Execute code

```
for(var n = 0; n < 3; n=n+1 ){  
    alert("hello user")  
}
```





Loops

- 1. Declare new variable
- 2. Conditional (based on said variable)
- 3. Update variable
- 4. Execute code

```
for(var n = 0; n < 3; n=n+1,  
    alert("hello user")  
}
```





Loops

- 1. Declare new variable
- 2. Conditional (based on said variable)
- 3. Update variable
- 4. Execute code

```
for(var n = 0; n < 3; n=n+1 ,  
    alert("hello user")  
}
```





Loops

- 1. Declare new variable
- 2. Conditional (based on said variable)
- 3. Update variable
- 4. Execute code

```
for(var n = 0; n < 3; n=n+1  
    alert("hello user")  
}
```





Loops

- 1. Variable gets declared
- 2. Conditional is checked
- 3. Code executes
- 4. Variable is updated

```
for(var n = 0; n < 3; n=n+1 ){  
    alert("hello user")  
}
```





Loops

- This for loop pattern is particularly useful for doing the same thing, N times.
- Let's actually use this pattern to make a website
- Consider this problem





Loops

- Build a website:
 - Asks the user how many items they want
 - Have them enter those items
- Builds an HTML list of those items
 - Maybe we make it fancier based on time





Loops

- End up with something like this
- Not a single canonical solution
- + ● Goes for all of programming

```
document.write("<ul>");

var numItems = prompt("How many items?");

numItems = parseInt(numItems);

for(var i = 0; i < numItems; i = i + 1){
    var curItem = prompt("Enter the next item: ");

    document.write("<li>" + curItem + "</li>");
}

document.write("</ul>")
```